Experiment 6: Circular Linked List Operations

Problem Statement:
WACP to implement polynomial operations using CLL. The prog should support the following operations:
i. Create a polynomial by inserting terms
ii. Display the polynomial in a readable mathematical form
iii. Add two polynomials and display the resultant polynomial.

Objective:
i. To implement a polynomial representation using CLL by dynamically creating and sorting polynomial terms.
ii. To perform fundamental polynomial operations such as creating, displaying, and adding two polynomial using CLL techniques.
iii. To understand advantages of CLL in managing dynamic data structures & efficiently handling polynomial arithmetic by combining like terms.

Theory:
A polynomial is a mathematical expression made up of terms, where each term consists of a coeff. & a var raised to a non-negative int exponent. A general poly. can be represented as

$$P(x) = a_n x^n + a_{n-1} x^{n-1} + \ldots + a_1 x + a_0$$

where $a_n, a_{n-1}, \ldots, a_1, a_0$ are coeff. & n is the highest exponent of the polynomial. For eg.

$$P(x) = 5x^3 + 2x^2 - 4x + 7$$

In a computer prog, each polynomial term can be represented using two fields: coeff & exponent. Since the nos of terms in a poly. may vary, a dynamic ds such as LL is suitable for sorting polynomial terms.

A CLL is a variation of a LL in which the last node points back to the first node instead of containing a NULL pointer. This creates a circular structure. Each node in the CLL contains the coeff, exponent, & a pointer to the next node. The circular nature of the list is useful for repeated traversal & processing of poly terms w/o requiring a seprate NULL cond.
For poly. represent^{n}, the terms are generally stored in desc. order of exponents. For eg, the polynomial:

$$6x^4 + 3x^2 + 2x + 1$$

can be stored as:

$$(6,4) \longrightarrow (3,2) \longrightarrow (2,1) \longrightarrow (1,0)$$

where each pair represents (coeff, exponent). The last node is linked back to the first node forming a CLL.
The creation opr^{n} inserts poly. terms into the CLL. While inser-ting, the prog should maintain the order of exponents & avoid storing unnecessary terms with a coeff. of zero. If a term w/ an existing expo. is inserted, its coeff. can be combined w/ the existing term.
The display opern traverses the CLL and prints each term in a readable mathematical format. Special cases such as exponent zero, exponent one, +ve & -ve coeff., & coeff equal to one must be handled properly. For eg., a node containing coeff. 3 & expo 2 is displayed as 3x^2, while a node containing coeff 5 & expo 0 is displayed as 5.
Poly. add is performed by comparing the exponents of terms from two poly. If the expo are equal, the coeff are added & the resulting term is inserted into the resultant poly. If the expo of one term is greater than the other, then term is directly copied to the resultant poly. This process continues until all terms from both poly. are processed. Terms having a resultant coeff. of zero are not included in the final polynomial.

Test cases:

Test Case	Test Scenario/Condⁿ	Operations	Expected output
TC1	Empty polynomial	Total terms = 0	0
TC2	Single Term Polynomial	Terms = 1 (C:5, E:2)	5x^2
TC3	Multiple Terms Seq.	Terms = 3 (3,3),	3x^3+2x^2+1
	Creation	(2,2), (1,0)	
TC4	Addition	P1: 3x^3+2x^2+5	7x^3+7x^2+7
		P2: 4x^3+5x^2+2	
TC5	Addition	P1: 5x^4 + 2x^2	5x^4+3x^3+2x^2+4x
		P2: 3x^3 + 4x	
TC6	Addition	P1: 4x^3 - 2x^2+5	2
		P2: -4x^3+2x^2-3	
TC7	Repeated Traversall	display_polynomial()	Output 1: 3x^2+4
	Display	twice on 3x^2+4	Output 2: 3x^2+4

Pseudo Codes:

```
Structure PolyNode:
        int coeff
        int exp
        pointer next
end Structure

Algorithm insertTerm(head, coeff, exp):
        if coeff == 0:
            return head
        create new_node
        set new_node.coeff = coeff
        set new_node.exp = exp
        if head == NULL:
            set new_node.next = new_node
```

```
            set head = new_node
            return head
        set temp = head
        while temp.next != head:
            temp = temp.next
        set temp.next = new_node
        set new_node.next = head
        return head
end insertTerm

Algorithm create_polynomial():
        set head = NULL
        read total nos of terms n
        for i = 1 to n:
            read coeff and exp for term i
            set head = insertTerm(head, coeff, exp)
        return head
End create_polynomial

Algorithm display_polynomial(head):
        if head == NULL:
            display "0"
            return
        set ptr = head
        set isFirst = true
        repeat:
            if ptr.coeff > 0 and isFirst == false:
                print "+"
            else if ptr.coeff < 0:
                print "-"
```

```
        set absCoeff = absolute(ptr.coeff)
        if absCoeff != 1 or ptr.exp == 0:
            print absCoeff
        if ptr.exp > 0:
            print "x"
            if ptr.exp > 1:
                print "^" + ptr.exp
        set isfirst = false
        set ptr = ptr.next
    while ptr != head
    display newline
end display-polynomial
```

```
Adgorithm addPolynomial(poly1, poly2):
    if poly1 == NULL : Return poly 2
    if poly 2 == NULL : Return poly 1
    set poly3 = NULL
    set p1 = poly 1
    set p 2= poly 2
    set done 1 = false; done 2 = false
    while done1 == false or done2 == false:
        if done1 ==false and done2 == false:
            if p1.exp > p2.exp:
                set poly3 = insertTerm(poly3, p1.coeff, p1.exp)
                p1 = p1.next
                if p1 == poly1 : set done1 = true
            else if p2.exp > p1.exp:
                set poly3 = insertTerm(poly3, p2.coeff, p2.exp)
                p2 = p2.next
                if p2 == poly2 : set done2 = true
```

```
        else (p1.exp == p2.exp):
            set sumCoeff = p1.coeff + p2.coeff
            if sumCoeff != 0:
                set poly3 = insertTerm(poly3, sumCoeff, p1.exp)
            p1 = p1.next
            p2 = p2.next
            if p1 == poly1: set done1 = true
            if p2 == poly2: set done2 = true
    else if done1 == false:
        set poly3 = insertTerm(poly3, p1.coeff, p1.exp)
        p1 = p1.next
        if p1 == poly1 : set done1 = true
    else if done2 == false:
        set poly3 = insertTerm(poly3, p2coeff, p2.exp)
        p2 = p2.next
        if p2 == poly2 : set done = true
    return poly3
enel addPolynomial
```

Time Complexity & Space complexity:

create_polynomial()	$O(N^2)$ or $O(N)$	$O(N)$
display_polynomial()	$O(N)$	$O(1)$
addPolynomial()	$O(M+N$	$O(M+N)$

Conclusion:

The prog. successfully implements poly. creation, display, & addition using a CLL, demonstrating efficient dynamic memory management & polynomial manipulation.

FAQs:

1. Why is a CLL used to represent poly. instead of a LLL?

Ans. A CLL avoids encountering NULL pointers during poly traversal, enabling smooth retraversals & continuous operations w/o edge-case crashes. Tracking the list w/ a single pointer to the last node allows O(1) access to both the start (head) & tail for appending new terms.

2. What info is stored in each node of CLL?

Ans. It stores the numerical coeff, the term's exp & a pointer (next) containing the memory address of the next term in the circular list.

3. What happens if one of the poly. is empty during the add. opern?

Ans. If one of the poly is empty, the add algorithm bypasses term-by-term comparisons & directly returns the non-empty poly as the result.

Code:

```
#include <stdio.h>
#include <stdlib.h>

struct PolyNode {
    int coeff;
    int exp;
    struct PolyNode *next;
};

struct PolyNode *insertTerm(struct PolyNode *head, int
coeff, int exp) {
    if (coeff == 0) return head;
    struct PolyNode *new_node = (struct PolyNode *)
malloc(sizeof(struct PolyNode));
    new_node->coeff = coeff;
    new_node->exp = exp;
    if (head == NULL) {
        new_node->next = new_node;      /* points to itself */
        return new_node;
    }
    struct PolyNode *temp = head;
    while (temp->next != head) temp = temp->next;
    temp->next = new_node;
    new_node->next = head;
    return head;
}

struct PolyNode *create_polynomial() {
    struct PolyNode *head = NULL;
```

```
    int n, coeff, exp;
    printf("Total number of terms: ");
    scanf("%d", &n);
    for (int i = 1; i <= n; i++) {
        printf("Term %d (coeff exp): ", i);
        scanf("%d %d", &coeff, &exp);
        head = insertTerm(head, coeff, exp);
    }
    return head;
}

void display_polynomial(struct PolyNode *head) {
    if (head == NULL) { printf("0\n"); return; }
    struct PolyNode *ptr = head;
    int isFirst = 1;
    do {
        if (ptr->coeff > 0 && isFirst == 0) printf(" + ");
        else if (ptr->coeff < 0) printf(isFirst ? "-" : " - ");
        int absCoeff = abs(ptr->coeff);
        if (absCoeff != 1 || ptr->exp == 0) printf("%d", absCoeff);
        if (ptr->exp > 0) {
            printf("x");
            if (ptr->exp > 1) printf("^%d", ptr->exp);
        }
        isFirst = 0;
        ptr = ptr->next;
    } while (ptr != head);
    printf("\n");
}
```

```
struct PolyNode *addPolynomial(struct PolyNode *poly1,
struct PolyNode *poly2) {
    if (poly1 == NULL) return poly2;
    if (poly2 == NULL) return poly1;
    struct PolyNode *poly3 = NULL, *p1 = poly1, *p2 = poly2;
    int done1 = 0, done2 = 0;
    while (done1 == 0 || done2 == 0) {
        if (done1 == 0 && done2 == 0) {
            if (p1->exp > p2->exp) {
                poly3 = insertTerm(poly3, p1->coeff, p1->exp);
                p1 = p1->next;
                if (p1 == poly1) done1 = 1;
            } else if (p2->exp > p1->exp) {
                poly3 = insertTerm(poly3, p2->coeff, p2->exp);
                p2 = p2->next;
                if (p2 == poly2) done2 = 1;
            } else {
                int sumCoeff = p1->coeff + p2->coeff;
                if (sumCoeff != 0) poly3 = insertTerm(poly3,
sumCoeff, p1->exp);
                p1 = p1->next;
                p2 = p2->next;
                if (p1 == poly1) done1 = 1;
                if (p2 == poly2) done2 = 1;
            }
        } else if (done1 == 0) {
            poly3 = insertTerm(poly3, p1->coeff, p1->exp);
            p1 = p1->next;
            if (p1 == poly1) done1 = 1;
        } else {
```

```
            poly3 = insertTerm(poly3, p2->coeff, p2->exp);
            p2 = p2->next;
            if (p2 == poly2) done2 = 1;
        }
    }
    return poly3;
}

int main() {
    struct PolyNode *poly1 = NULL, *poly2 = NULL, *poly3 =
NULL;
    int ch;
    do {
        printf("\n1.Create P1 2.Create P2 3.Display 4.Add
5.Exit\nChoice: ");
        scanf("%d", &ch);
        switch (ch) {
            case 1: printf("Enter terms in descending order of
exp\n");
                    poly1 = create_polynomial(); break;
            case 2: printf("Enter terms in descending order of
exp\n");
                    poly2 = create_polynomial(); break;
            case 3: printf("P1 = "); display_polynomial(poly1);
                    printf("P2 = "); display_polynomial(poly2); break;
            case 4: poly3 = addPolynomial(poly1, poly2);
                    printf("P1 + P2 = "); display_polynomial(poly3);
break;
            case 5: printf("Exiting\n"); break;
            default: printf("Invalid choice\n");
```

```
        }
    } while (ch != 5);
    return 0;
}
```

Input and Output:

TC1:

1

0

3

```
C:\Users\shrey\CLionProjects\untitled\Ass1.exe

1.Create P1
2.Create P2
3.Display
4.Add
5.Exit
Choice:1

Enter terms in descending order of exp
Total number of terms:0


1.Create P1
2.Create P2
3.Display
4.Add
5.Exit
Choice:3

P1 = 0
P2 = 0
```

TC2:

1

1

5 2

3

```
1.Create P1
2.Create P2
3.Display
4.Add
5.Exit
Choice:1

Enter terms in descending order of exp
Total number of terms:1

Term 1 (coeff exp):5 2


1.Create P1
2.Create P2
3.Display
4.Add
5.Exit
Choice:3

P1 = 5x^2
P2 = 0
```

TC3:

1

3

3 3

2 2

1 0

0

```
C:\Users\shrey\CLionProjects\untitled\Ass1.exe

1.Create P1
2.Create P2
3.Display
4.Add
5.Exit
Choice:1

Enter terms in descending order of exp
Total number of terms:3

Term 1 (coeff exp):3 3

Term 2 (coeff exp):2 2

Term 3 (coeff exp):1 0


1.Create P1
2.Create P2
3.Display
4.Add
5.Exit
Choice:3

P1 = 3x^3 + 2x^2 + 1
P2 = 0
```

TC4:

1

3

3 3

2 2

5 0

2

3

4 3

5 2

2 0

4

```
C:\Users\shrey\CLionProjects\untitled\Ass1.exe

1.Create P1
2.Create P2
3.Display
4.Add
5.Exit
Choice:1

Enter terms in descending order of exp
Total number of terms:3

Term 1 (coeff exp):3 3

Term 2 (coeff exp):2 2

Term 3 (coeff exp):1 0


1.Create P1
2.Create P2
3.Display
4.Add
5.Exit
Choice:3

P1 = 3x^3 + 2x^2 + 1
P2 = 0
```

```
1.Create P1
2.Create P2
3.Display
4.Add
5.Exit
Choice: 1

Enter terms in descending order of exp
Total number of terms:3

Term 1 (coeff exp):3 3

Term 2 (coeff exp):2 2

Term 3 (coeff exp):5 0


1.Create P1
2.Create P2
3.Display
4.Add
5.Exit
Choice:2

Enter terms in descending order of exp
Total number of terms:3
```

```
Term 1 (coeff exp):4 3

Term 2 (coeff exp):5 2

Term 3 (coeff exp):2 0


1.Create P1
2.Create P2
3.Display
4.Add
5.Exit
Choice:4

P1 + P2 = 7x^3 + 7x^2 + 7
```

TC5:

1

2

5 4

2 2

2

2

3 3

4 1

4

```
1.Create P1
2.Create P2
3.Display
4.Add
5.Exit
Choice:1

Enter terms in descending order of exp
Total number of terms:2

Term 1 (coeff exp):5 4

Term 2 (coeff exp):2 2


1.Create P1
2.Create P2
3.Display
4.Add
5.Exit
Choice:2

Enter terms in descending order of exp
Total number of terms:2

Term 1 (coeff exp):3 3

Term 2 (coeff exp):4 1
```

```
1.Create P1
2.Create P2
3.Display
4.Add
5.Exit
Choice:4

P1 + P2 = 5x^4 + 3x^3 + 2x^2 + 4x
```

TC6:

1

3

4 3

-2 2

5 0

2

3

-4 3

2 2

-3 0

4

```
1.Create P1
2.Create P2
3.Display
4.Add
5.Exit
Choice:1

Enter terms in descending order of exp
Total number of terms:3

Term 1 (coeff exp):4 3

Term 2 (coeff exp):-2 2

Term 3 (coeff exp):5 0


1.Create P1
2.Create P2
3.Display
4.Add
5.Exit
Choice:2

Enter terms in descending order of exp
Total number of terms:3
```

```
Term 1 (coeff exp):-4 3

Term 2 (coeff exp):2 2

Term 3 (coeff exp):-3 0


1.Create P1
2.Create P2
3.Display
4.Add
5.Exit
Choice:4

P1 + P2 = 2
```

TC7:

1

2

3 2

4 0

3

3

```
1.Create P1
2.Create P2
3.Display
4.Add
5.Exit
Choice:1

Enter terms in descending order of exp
Total number of terms:2

Term 1 (coeff exp):3 2

Term 2 (coeff exp):4 0


1.Create P1
2.Create P2
3.Display
4.Add
5.Exit
Choice:3

P1 = 3x^2 + 4
P2 = -4x^3 + 2x^2 - 3
```

```
1.Create P1
2.Create P2
3.Display
4.Add
5.Exit
Choice:3

P1 = 3x^2 + 4
P2 = -4x^3 + 2x^2 - 3

1.Create P1
2.Create P2
3.Display
4.Add
5.Exit
Choice:6

Invalid choice

1.Create P1
2.Create P2
3.Display
4.Add
5.Exit
Choice:5

Exiting
```